

IOMANIP_REFERENCE

Core Definition

Manipulators are objects/functions inserted into a stream with `<<` or `>>` that change stream formatting state rather than transferring data. `<iomanip>` provides the *parameterized* ones (`setw`, `setprecision`, `setfill`, ...); the *unparameterized* ones (`hex`, `fixed`, `boolalpha`, ...) live in `<ios>` and `<ostream>` and arrive via `<iostream>`.

#CPP #IOMANIP #MANIPULATORS #FORMATTING #SETW #SETPRECISION #ALIGNMENT

COMPLETE MANIPULATOR QUICK REFERENCE

PARAMETERIZED MANIPULATORS — `<iomanip>`

```
// =====
// FIELD WIDTH, FILL & PRECISION
// =====
std::setw(n)           // int      | Set field width for NEXT item | Sticky for ONE operation only
std::setfill(ch)       // char    | Set padding character        | PERSISTENT until changed
std::setprecision(n)   // int      | Set decimal precision        | PERSISTENT; meaning depends on
    float mode          //          |                               |
                        //          | default mode: n = significant digits (default 6)
                        //          | fixed/scientific: n = digits after the point

// =====
// FLAG MANIPULATION (raw fmtflags control)
// =====
std::setiosflags(mask) // fmtflags | Set the given flags          | Equivalent to os.setf(mask)
std::resetiosflags(mask) // fmtflags | Clear the given flags        | Equivalent to os.unsetf(mask)
std::setbase(base)     // int      | Set numeric base             | 8=oct, 10=dec, 16=hex; other → dec

// =====
// QUOTED STRINGS (C++14)
// =====
std::quoted(str)        // string   | Quote/unquote a string      | Output: wraps in " and escapes; Input:
    parses quoted
std::quoted(str, delim) // +delim   | Custom delimiter             | Default delim '"'
std::quoted(str, delim, esc) // +escape  | Custom escape char           | Default escape '\\'
                        //          | Round-trips strings containing spaces and quotes

// =====
// TIME & MONEY (locale-dependent)
// =====
std::put_time(tmb, fmt) // tm*+fmt  | Format a calendar time       | strftime-style format string
std::get_time(tmb, fmt) // tm*+fmt  | Parse a calendar time        | Input manipulator
std::put_money(mon)     // value     | Format monetary value        | Uses locale money_put facet
std::put_money(mon, intl) // +bool     | International format        | true → e.g. "USD 1.00"
std::get_money(mon)     // value&    | Parse monetary value         | Input manipulator
std::get_money(mon, intl) // +bool     | Parse international          | Input manipulator
```

UNPARAMETERIZED MANIPULATORS — `<ios>` / `<ostream>` (via `<iostream>`)

```
// =====
// NUMERIC BASE (persistent; mutually exclusive)
// =====
std::dec                //          | Decimal output/input (DEFAULT)
std::oct                //          | Octal
```

```

std::hex                // Hexadecimal
std::showbase           // Prefix: 0x for hex, 0 for octal
std::noshowbase         // No prefix (DEFAULT)
std::uppercase          // 0X1A, 1.5E+10, INF
std::nouppercase        // 0x1a, 1.5e+10, inf (DEFAULT)

// =====
// FLOATING-POINT FORMAT (persistent; mutually exclusive)
// =====
std::fixed              // Fixed-point: 3.141593
std::scientific         // Scientific: 3.141593e+00
std::hexfloat           // Hexadecimal float (C++11): 0x1.91eb86p+1
std::defaultfloat       // General/shortest form (C++11) – the DEFAULT
std::showpoint          // Always print the decimal point: 5.00000
std::noshowpoint        // Omit if no fractional part (DEFAULT): 5

// =====
// ALIGNMENT (persistent; needs setw to be visible)
// =====
std::left               // Pad on the right |value|
std::right              // Pad on the left |value| (DEFAULT)
std::internal           // Pad after sign / base prefix |- 42| |0x 1A|

// =====
// SIGN & BOOLEAN
// =====
std::showpos            // Prefix + on non-negative numbers
std::noshowpos          // No + (DEFAULT)
std::boolalpha          // Print/parse bools as true / false
std::noboolalpha        // Print/parse bools as 1 / 0 (DEFAULT)

// =====
// WHITESPACE & FLUSHING
// =====
std::skipws             // Skip leading whitespace on >> (DEFAULT)
std::noskipws           // Do NOT skip whitespace on >> (so >> ch reads spaces)
std::ws                // Input manip: consume whitespace NOW (is >> std::ws)
std::endl              // Insert '\n' AND flush
std::ends              // Insert '\0'
std::flush             // Flush the buffer
std::unitbuf           // Flush after every output operation
std::nounitbuf         // Buffer normally (DEFAULT for cout)
std::flush_emit        // C++20: flush and emit (for syncstreams)

// =====
// C++20 – SYNCHRONIZED / EMIT
// =====
std::emit_on_flush     // ostream: transmit on flush
std::noemit_on_flush   // ostream: do not transmit on flush (DEFAULT)

```

FLAG MASK GROUPS

```

std::ios::basefield     // = dec | oct | hex
std::ios::floatfield    // = fixed | scientific
std::ios::adjustfield   // = left | right | internal

// Correct way to swap one flag within a group:
os.setf(std::ios::hex, std::ios::basefield); // clears dec/oct first
os.setf(std::ios::left, std::ios::adjustfield);
os.setf(std::ios::fixed, std::ios::floatfield);

```

STICKINESS TABLE — the single most common source of bugs

Manipulator	Scope
setw(n)	ONE operation, then resets to 0
setfill(ch)	Persistent
setprecision(n)	Persistent
hex / oct / dec	Persistent
fixed / scientific	Persistent
left / right/internal	Persistent
boolalpha	Persistent
showpos / showbase	Persistent
endl / flush / ws	One-shot action, no state change

COMMON PATTERNS & EXAMPLES

Aligned Table Output

```
#include <iostream>
#include <iomanip>
#include <string>

struct Row { std::string name; int qty; double price; };

int main() {
    Row rows[] = {"Widget", 12, 4.5}, {"Gadget", 3, 129.99}, {"Doohickey", 250, 0.75};

    std::cout << std::left << std::setw(14) << "Item"
              << std::right << std::setw(6) << "Qty"
              << std::setw(12) << "Price" << '\n'
              << std::string(32, '-') << '\n';

    std::cout << std::fixed << std::setprecision(2);
    for (const auto& r : rows) {
        std::cout << std::left << std::setw(14) << r.name
                  << std::right << std::setw(6) << r.qty
                  << std::setw(12) << r.price << '\n';
    }
}
// Item          Qty      Price
// -----
// Widget         12        4.50
// Gadget          3       129.99
// Doohickey      250         0.75
```

Leading Zeros

```
std::cout << std::setfill('0') << std::setw(5) << 42 << '\n'; // 00042
std::cout << std::setfill(' '); // Reset – setfill is sticky!

// Timestamp
int h = 9, m = 5, s = 3;
std::cout << std::setfill('0')
          << std::setw(2) << h << ':'
          << std::setw(2) << m << ':'
          << std::setw(2) << s << '\n'; // 09:05:03
```

Hex Dump

```
#include <iostream>
#include <iomanip>

void hexdump(const unsigned char* data, std::size_t n) {
    std::cout << std::hex << std::setfill('0');
    for (std::size_t i = 0; i < n; ++i) {
        if (i % 16 == 0) std::cout << std::setw(8) << i << " ";
        std::cout << std::setw(2) << static_cast<int>(data[i]) << " ";
        if (i % 16 == 15) std::cout << '\n';
    }
    std::cout << std::dec << std::setfill(' ') << '\n'; // Restore
}
```

Precision: Significant Digits vs Decimals

```
double pi = 3.14159265358979;

std::cout << std::setprecision(4) << pi << '\n'; // 3.142 (4 significant digits)
std::cout << std::fixed << std::setprecision(4) << pi << '\n'; // 3.1416 (4 decimals)
std::cout << std::scientific << std::setprecision(4) << pi << '\n'; // 3.1416e+00
std::cout << std::defaultfloat; // Back to default mode
```

Currency-Style Column

```
std::cout << std::fixed << std::setprecision(2)
          << std::right << std::setw(12) << 1234.5 << '\n'; //      1234.50

// With internal alignment and a sign:
std::cout << std::internal << std::showpos << std::setfill('.')
          << std::setw(12) << -1234.5 << '\n'; // -....1234.50
```

Saving and Restoring Format State

```
#include <iostream>
#include <ios>

// Manual save/restore
void printHex(std::ostream& os, int v) {
    std::ios::fmtflags oldFlags = os.flags();
    char oldFill = os.fill();
    std::streamsize oldPrec = os.precision();

    os << std::hex << std::showbase << v;

    os.flags(oldFlags);
    os.fill(oldFill);
    os.precision(oldPrec);
}

// RAII version – preferred in library code
class FormatGuard {
    std::ios& s_;
    std::ios::fmtflags f_;
    char fill_;
    std::streamsize prec_;
public:
    explicit FormatGuard(std::ios& s)
        : s_(s), f_(s.flags()), fill_(s.fill()), prec_(s.precision()) {}
    ~FormatGuard() { s_.flags(f_); s_.fill(fill_); s_.precision(prec_); }
    FormatGuard(const FormatGuard&) = delete;
    FormatGuard& operator=(const FormatGuard&) = delete;
};
```

```
void safePrint(std::ostream& os, double v) {
    FormatGuard guard(os);
    os << std::fixed << std::setprecision(8) << v << '\n';
} // Original formatting restored automatically

// Standard alternative: copy all formatting from a pristine stream
// os.copyfmt(std::ios(nullptr));
```

Quoted Strings — Round-Tripping (C++14)

```
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>

int main() {
    std::string original = R"(He said "hi" to me)";

    std::stringstream ss;
    ss << std::quoted(original);
    std::cout << ss.str() << '\n'; // "He said \"hi\" to me"

    std::string roundtrip;
    ss >> std::quoted(roundtrip);
    std::cout << (roundtrip == original) << '\n'; // 1

    // Without quoted(), >> would stop at the first space.
}
```

Date/Time Formatting

```
#include <iostream>
#include <iomanip>
#include <ctime>

int main() {
    std::time_t t = std::time(nullptr);
    std::tm tmv = *std::localtime(&t);

    std::cout << std::put_time(&tmv, "%Y-%m-%d %H:%M:%S") << '\n';
    std::cout << std::put_time(&tmv, "%A, %B %d, %Y") << '\n';

    // Parsing
    std::tm parsed{};
    std::istringstream in("2026-09-08 14:30:00");
    in >> std::get_time(&parsed, "%Y-%m-%d %H:%M:%S");
    if (in.fail()) std::cerr << "Parse failed\n";
}
```

Common `put_time` / `get_time` specifiers

%Y year (2026)	%m month 01-12	%d day 01-31
%y 2-digit year	%B full month name	%b abbrev month
%H hour 00-23	%M minute 00-59	%S second 00-60
%I hour 01-12	%p AM/PM	%j day of year
%A full weekday	%a abbrev weekday	%Z timezone name
%F = %Y-%m-%d	%T = %H:%M:%S	%c locale date+time
%e day, space-padded	%z UTC offset	%% literal %

Boolean and Base Display

```
std::cout << std::boolalpha << (1 < 2) << '\n';           // true
std::cout << std::hex << std::showbase << std::uppercase << 255; // 0XFF
std::cout << std::dec << std::noshowbase << std::nouppercase; // Restore
```

Reading Whitespace-Sensitive Input

```
char c;
std::cin >> std::noskipws;           // Now >> reads spaces and newlines too
while (std::cin >> c) { /* process every character */ }
std::cin >> std::skipws;             // Restore
```

Progress / Bar Rendering

```
void bar(double frac, int width = 40) {
    int filled = static_cast<int>(frac * width);
    std::cout << '[' << std::setfill('#') << std::setw(filled) << ""
               << std::setfill('.') << std::setw(width - filled) << "" << "]" << " "
               << std::fixed << std::setprecision(1) << frac * 100 << "%\r"
               << std::flush;
    std::cout << std::setfill(' ');
}
```

IMPORTANT CONCEPTS

setw Applies to the Next Item Only

```
std::cout << std::setw(10) << "a" << "b" << '\n'; //      ab
std::cout << std::setw(10) << "a" << std::setw(10) << "b" << '\n'; //      a      b
```

Repeat `setw` for every field you want padded.

Precision Means Two Different Things

In **default** (`defaultfloat`) mode, precision counts *significant digits*. In **fixed** or **scientific** mode it counts *digits after the decimal point*. Setting **fixed** before **setprecision** is what most table code actually wants.

Alignment Needs Width

left / **right** / **internal** do nothing unless **setw(n)** gives the field room to pad into.

internal Is for Signs and Prefixes

It pads between the sign (or `0x`) and the digits — useful for aligning negative numbers or hex values in a column.

Manipulators Are Just Functions

`std::hex` is a function taking and returning `ios_base&`. `operator<<` has an overload for such function pointers, which calls them on the stream. Writing your own is straightforward:

```
std::ostream& tab(std::ostream& os) { return os << '\t'; }
std::cout << 1 << tab << 2 << '\n';
```

A parameterized one needs a helper object with an `operator<<`:

```
struct Indent { int n; };
inline Indent indent(int n) { return Indent{n}; }
inline std::ostream& operator<<(std::ostream& os, Indent i) {
    return os << std::setfill(' ') << std::setw(i.n) << " ";
}
```

copyfmt Copies Everything

`os.copyfmt(other)` copies flags, precision, width, fill, locale, tied stream and user words — but **not** the buffer or the error state. `os.copyfmt(std::ios(nullptr))` resets a stream to default formatting in one line.

<format> and <print> Are the Modern Alternative

For new code, C++20's `std::format` and C++23's `std::print` express the same intent far more readably and without sticky state:

```
#include <format>
#include <print> // C++23
std::cout << std::format("{:<14}{:>6}{:>12.2f}\n", name, qty, price);
std::println("{:#06x}", 255); // 0x00ff
```

Manipulators remain necessary when interacting with existing stream-based APIs, custom `operator<<`, or when the target is any `std::ostream`.

FORMAT SPEC QUICK COMPARISON (streams vs `std::format`)

Intent	stream manipulators	<code>std::format</code>
width 8, right	<code>std::setw(8) << v</code>	<code>"{:>8}"</code>
width 8, left	<code>std::left << std::setw(8) << v</code>	<code>"{:<8}"</code>
zero-pad to 5	<code>std::setfill('0') << std::setw(5) << v</code>	<code>"{:05}"</code>
2 decimals	<code>std::fixed << std::setprecision(2)</code>	<code>"{: .2f}"</code>
hex with prefix	<code>std::hex << std::showbase</code>	<code>"{:#x}"</code>
scientific	<code>std::scientific << std::setprecision(3)</code>	<code>"{: .3e}"</code>
bool as text	<code>std::boolalpha</code>	<code>"{}"</code> (default)
sign always	<code>std::showpos</code>	<code>"{:+}"</code>

BEST PRACTICES

1. Set **fixed** before **setprecision** when you want N decimal places
2. Reset **setfill** after using a non-space fill — it is sticky and will corrupt later output
3. Repeat **setw** per field — it never persists
4. Use **RAII** (or **copyfmt**) to restore format state in any function that takes an `ostream&`
5. Prefer **setf(flag, mask)** over bare **setf(flag)** when switching within a group
6. Use **std::quoted** for any string round-trip through a stream
7. Don't mix **hex** with **signed negative values** unless you understand the conversion — cast to an unsigned type first
8. Reach for **std::format** / **std::print** in new code; keep manipulators for stream-based interfaces
9. **put_time** needs a valid **tm** — zero-initialize (`std::tm t{};`) before **get_time**
10. Test alignment with real data widths, not just short samples

RELATED HEADERS

```
#include <iomanip>    // setw, setfill, setprecision, quoted, put_time, put_money
#include <ios>        // hex, fixed, left, boolalpha, showbase ... + ios_base
#include <ostream>    // endl, ends, flush, emit_on_flush
#include <istream>    // ws
#include <format>     // C++20 std::format / format_to
#include <print>      // C++23 std::print / println
#include <locale>     // facets behind put_money/put_time
```

EXTERNAL RESOURCES

- **<iomanip>**: <https://en.cppreference.com/w/cpp/header/iomanip>
- **Manipulator list**: <https://en.cppreference.com/w/cpp/io/manip>
- **std::format spec**: <https://en.cppreference.com/w/cpp/utility/format/spec>

Standard: C++11 / C++14 (`quoted`) / C++20 (`emit_on_flush`) / C++23 (`print`) **Last Updated:** September 2026